

TP17 — Déployer une configuration ACI complète via script

Formation NetOps : Automatiser la gestion de son Infrastructure Réseau — Ambient IT

Description

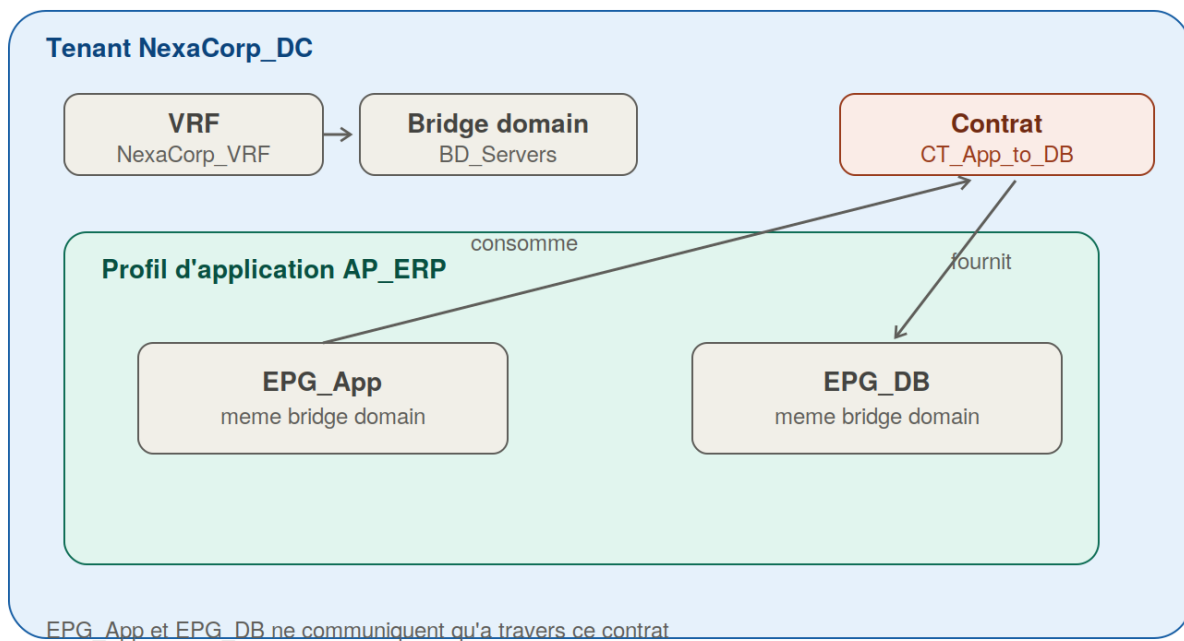


Figure 1: Arbre de configuration ACI à déployer

Cisco ACI (APIC) gère le datacenter comme un arbre d'objets (tenant, VRF, bridge domain, profils d'application, EPG, contrats), et une bonne pratique consiste à déployer tout cet arbre en un seul appel plutôt qu'objet par objet. Ce TP simule ce modèle, déploie un tenant complet pour NexaCorp (deux EPG représentant les niveaux applicatif et base de données, reliés par un contrat), et vérifie une règle fondamentale d'ACI : sans contrat explicite entre deux EPG, tout trafic entre eux est refusé par défaut.

Prérequis

- TP1 à TP16 validés

Étapes

Étape 1 — Créer sa branche

```
cd netops-project-<nom-groupe>
git checkout dev
git pull origin dev
git checkout -b <prenom>-<nom>-jour5-tp17
```

Étape 2 — Créer le mock APIC (contrôleur ACI)

```
cat > scripts/mock_apic_api.py << 'EOF'
from flask import Flask, request, jsonify

app = Flask(__name__)

VALID_USER = "admin"
VALID_PASS = "acipass123"
VALID_TOKEN = "apic-fake-token-xyz"

TENANTS = {}

def check_token():
    return request.headers.get("APIC-Cookie") == VALID_TOKEN

def find_epgs(tenant_body):
    """Parcourt l'arbre poste pour retrouver les EPG et leurs contrats."""
    epgs = {}
    for ap in tenant_body.get("fvTenant", {}).get("children", []):
        if "fvAp" not in ap:
            continue
        for epg_wrapper in ap["fvAp"].get("children", []):
            if "fvAEPg" not in epg_wrapper:
                continue
            epg = epg_wrapper["fvAEPg"]
            name = epg["attributes"]["name"]
            contracts = {"consumes": [], "provides": []}
            for rel in epg.get("children", []):
                if "fvRsCons" in rel:
                    contracts["consumes"].append(rel["fvRsCons"]["attributes"]["tnVzBrCPName"])
                if "fvRsProv" in rel:
                    contracts["provides"].append(rel["fvRsProv"]["attributes"]["tnVzBrCPName"])
            epgs[name] = contracts
    return epgs
```

```

@app.route("/api/aaaLogin.json", methods=["POST"])
def aaa_login():
    body = request.get_json(force=True)
    creds = body.get("aaaUser", {}).get("attributes", {})
    if creds.get("name") != VALID_USER or creds.get("pwd") != VALID_PASS:
        return jsonify({"imdata": [{"error": {"attributes": {"text": "Unauthorized"}}}]}, 401)
    return jsonify({"imdata": [{"aaaLogin": {"attributes": {"token": VALID_TOKEN}}}]})

@app.route("/api/mo/uni/tn-<tenant_name>.json", methods=["POST"])
def deploy_tenant(tenant_name):
    if not check_token():
        return jsonify({"imdata": [{"error": {"attributes": {"text": "Missing or invalid APIC-Cookie"}}}]}, 401)
    body = request.get_json(force=True)
    if "fvTenant" not in body:
        return jsonify({"imdata": [{"error": {"attributes": {"text": "fvTenant racine requis"}}}]}, 400)

    TENANTS[tenant_name] = body
    return jsonify({"imdata": [{"fvTenant": {"attributes": {"name": tenant_name, "status": "created,modified"}}}]})

@app.route("/api/mo/uni/tn-<tenant_name>.json", methods=["GET"])
def get_tenant(tenant_name):
    if not check_token():
        return jsonify({"imdata": [{"error": {"attributes": {"text": "Missing or invalid APIC-Cookie"}}}]}, 401)
    if tenant_name not in TENANTS:
        return jsonify({"imdata": []}), 404
    return jsonify({"imdata": [TENANTS[tenant_name]]})

@app.route("/api/mo/uni/tn-<tenant_name>/contract-check.json", methods=["GET"])
def contract_check(tenant_name):
    """Endpoint de validation (non standard APIC, ajoute pour ce TP) :
    verifie qu'un contrat relie bien deux EPG donnees en parametres."""
    if not check_token():
        return jsonify({"imdata": [{"error": {"attributes": {"text": "Missing or invalid APIC-Cookie"}}}]}, 401)
    if tenant_name not in TENANTS:
        return jsonify({"error": f"Tenant {tenant_name} introuvable"}), 404

    consumer = request.args.get("consumer")
    provider = request.args.get("provider")
    epgs = find_epgs(TENANTS[tenant_name])

    if consumer not in epgs or provider not in epgs:
        return jsonify({"allowed": False, "reason": "EPG consumer ou provider introuvable"}), 404

    consumer_contracts = set(epgs[consumer]["consumes"])
    provider_contracts = set(epgs[provider]["provides"])

```

```

shared = consumer_contracts & provider_contracts

if shared:
    return jsonify({"allowed": True, "contract": list(shared)[0]})
return jsonify({"allowed": False, "reason": "Aucun contrat commun entre ces EPG (trafic refuse par défaut dans ACI)"})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=7000, debug=False)
EOF

```

ACI utilise un modèle d'authentification par cookie de session (APIC - Cookie), différent du header X-Auth-Token de DNA Center — chaque plateforme Cisco a ses propres conventions, d'où l'intérêt d'écrire un client dédié par produit plutôt qu'un client générique.

Étape 3 — Créer le client Python

```

cat > scripts/netops_client/apic_client.py << 'EOF'
import requests

class ACIError(Exception):
    """Erreur levee lors d'un echange avec l'API APIC (Cisco ACI)."""
    pass

class APICClient:
    """Client minimal pour l'API Cisco ACI (APIC) : authentication et deploiement de tenant."""

    def __init__(self, base_url, username, password, timeout=5):
        self.base_url = base_url.rstrip("/")
        self.username = username
        self.password = password
        self.timeout = timeout
        self.token = None

    def login(self):
        url = f"{self.base_url}/api/aaaLogin.json"
        body = {"aaaUser": {"attributes": {"name": self.username, "pwd": self.password}}}
        response = requests.post(url, json=body, timeout=self.timeout)
        if response.status_code != 200:
            raise ACIError(f"Authentication refusee ({response.status_code})")
        self.token = response.json()["imdata"][0]["aaaLogin"]["attributes"]["token"]
        return self.token

    def _headers(self):
        if not self.token:

```

```

        raise ACIError("Non authentifie : appelez login() d'abord")
    return {"APIC-Cookie": self.token}

def deploy_tenant(self, tenant_name, tenant_tree):
    url = f"{self.base_url}/api/mo/uni/tn-{tenant_name}.json"
    response = requests.post(url, headers=self._headers(), json=tenant_tree, timeout=self.timeout)
    if response.status_code != 200:
        raise ACIError(f"Echec du deploiement du tenant : {response.text}")
    return response.json()

def get_tenant(self, tenant_name):
    url = f"{self.base_url}/api/mo/uni/tn-{tenant_name}.json"
    response = requests.get(url, headers=self._headers(), timeout=self.timeout)
    if response.status_code != 200:
        raise ACIError(f"Tenant introuvable : {response.text}")
    return response.json()

def check_contract(self, tenant_name, consumer_epg, provider_epg):
    url = f"{self.base_url}/api/mo/uni/tn-{tenant_name}/contract-check.json"
    response = requests.get(
        url, headers=self._headers(), params={"consumer": consumer_epg, "provider": provider_epg}, timeout=self.timeout
    )
    return response.json()
EOF

cat > scripts/netops_client/__init__.py << 'EOF'
from .apic_client import APICClient, ACIError

__all__ = ["APICClient", "ACIError"]
EOF

```

Étape 4 — Créer le script de déploiement (arbre complet, un seul appel)

```

cat > scripts/deploy_aci_config.py << 'EOF'
import os
import sys

from netops_client import APICClient, ACIError

APIC_URL = os.environ.get("APIC_URL", "http://localhost:7000")
APIC_USER = os.environ.get("APIC_USER", "admin")
APIC_PASS = os.environ.get("APIC_PASS")

if not APIC_PASS:
    print("ERREUR : variable d'environnement APIC_PASS non definie.")
    sys.exit(1)

TENANT_NAME = "NexaCorp_DC"

```

```

TENANT_TREE = {
  "fvTenant": {
    "attributes": {"name": TENANT_NAME},
    "children": [
      {"fvCtx": {"attributes": {"name": "NexaCorp_VRF"}}},
      {
        "fvBD": {
          "attributes": {"name": "BD_Servers"},
          "children": [
            {"fvRsCtx": {"attributes": {"tnFvCtxName": "NexaCorp_VRF"}}}
          ],
        },
      },
      {
        "vzBrCP": {
          "attributes": {"name": "CT_App_to_DB"},
          "children": [
            {"vzSubj": {"attributes": {"name": "subj_app_db"}}}
          ],
        },
      },
      {
        "fvAp": {
          "attributes": {"name": "AP_ERP"},
          "children": [
            {
              "fvAEPg": {
                "attributes": {"name": "EPG_App"},
                "children": [
                  {"fvRsBd": {"attributes": {"tnFvBDName": "BD_Servers"}}},
                  {"fvRsCons": {"attributes": {"tnVzBrCPName": "CT_App_to_DB"}}},
                ],
              },
            },
            {
              "fvAEPg": {
                "attributes": {"name": "EPG_DB"},
                "children": [
                  {"fvRsBd": {"attributes": {"tnFvBDName": "BD_Servers"}}},
                  {"fvRsProv": {"attributes": {"tnVzBrCPName": "CT_App_to_DB"}}},
                ],
              },
            },
          ],
        },
      },
    ],
  },
},

```

```

    }
}

def main():
    client = APICClient(base_url=APIC_URL, username=APIC_USER, password=APIC_PASS)
    client.login()
    print("Authentifie sur APIC.\n")

    print(f"Deploiement du tenant '{TENANT_NAME}' (VRF, bridge domain, AP, 2 EPG, contrat)...")
    client.deploy_tenant(TENANT_NAME, TENANT_TREE)
    print("Deploiement reussi.\n")

    print("Verification : relecture du tenant deploye...")
    tenant = client.get_tenant(TENANT_NAME)
    if tenant["imdata"]:
        print(" OK : le tenant est bien present sur APIC.\n")
    else:
        print(" ECHEC : le tenant n'a pas ete trouve.\n")
        sys.exit(1)

    print("Verification metier : EPG_App peut-il consommer les services d'EPG_DB ?")
    result = client.check_contract(TENANT_NAME, consumer_epg="EPG_App", provider_epg="EPG_DB")
    if result.get("allowed"):
        print(f" OK : autorise via le contrat '{result['contract']}'.\n")
    else:
        print(f" REFUSE : {result.get('reason')}\n")
        sys.exit(1)

if __name__ == "__main__":
    try:
        main()
    except ACIError as e:
        print(f"Erreur ACI : {e}")
        sys.exit(1)

```

E0F

Notez la structure de `TENANT_TREE` : un dictionnaire Python qui reflète exactement l'arbre d'objets ACI (fvTenant contient des children, qui contiennent eux-mêmes des objets avec leurs propres children...). C'est le modèle "Managed Object" d'ACI — tout, y compris les relations entre objets (comme fvRsCons/fvRsProv pour les contrats), fait partie du même arbre poussé en un seul appel POST.

Étape 5 — Tester le déploiement complet

```
nohup python scripts/mock_apic_api.py > /tmp/apic.log 2>&1 &  
sleep 2  
export APIC_PASS=acipass123  
python scripts/deploy_aci_config.py
```

Résultat attendu :

Authentifie sur APIC.

Déploiement du tenant 'NexaCorp_DC' (VRF, bridge domain, AP, 2 EPG, contrat)...
Déploiement réussi.

Vérification : relecture du tenant deploye...

OK : le tenant est bien présent sur APIC.

Vérification métier : EPG_App peut-il consommer les services d'EPG_DB ?

OK : autorise via le contrat 'CT_App_to_DB'.

Étape 6 — Simuler un oubli de contrat (erreur de configuration courante en ACI)

Retirez temporairement la ligne fvRsCons (la relation "EPG_App consomme le contrat") de scripts/deploy_aci_config.py, dans la définition d'EPG_App :

```
# Supprimez temporairement cette ligne dans les "children" de EPG_App :  
{"fvRsCons": {"attributes": {"tnVzBrCPName": "CT_App_to_DB"}}},
```

Redémarrez le mock (nouveau tenant à déployer) et relancez le script :

```
python scripts/deploy_aci_config.py
```

Résultat attendu :

Déploiement réussi.

Vérification : relecture du tenant deploye...

OK : le tenant est bien présent sur APIC.

Vérification métier : EPG_App peut-il consommer les services d'EPG_DB ?

REFUSE : Aucun contrat commun entre ces EPG (trafic refuse par défaut dans ACI)

Le déploiement technique réussit (les objets existent bien), mais la vérification métier détecte l'erreur de conception : sans le contrat, ACI refuse tout trafic entre les deux EPG par défaut.

Étape 7 — Remettre la ligne fvRsCons

Restaurez la ligne supprimée à l'étape 6.

Étape 8 — Committer et pousser

```
git add scripts/mock_apic_api.py scripts/netops_client/apic_client.py scripts/netops_client/__init__.py scripts/deploy_aci_config.py
git commit -m "TP17 : déploiement d'une configuration ACI complete (tenant, EPG, contrat)"
git push -u origin <prenom>-<nom>-jour5-tp17
```

Ouvrir une pull request <prenom>-<nom>-jour5-tp17 → dev.

Comprendre le TP

Le modèle “objet géré” d’ACI est fondamentalement différent d’une configuration ligne par ligne façon CLI Cisco classique : tout est un arbre de tenant → VRF/bridge domain → profils d’application → EPG → relations. Déployer via script signifie construire cet arbre en mémoire (ici, un dictionnaire Python imbriqué) puis l’envoyer en un seul appel — plutôt que 10 appels séparés qui pourraient échouer indépendamment et laisser une configuration à moitié appliquée.

Les EPG et les contrats encodent la micro-segmentation qui fait la réputation d’ACI : par défaut, deux EPG ne peuvent pas communiquer, même sur le même bridge domain, tant qu’un contrat explicite ne les relie pas (l’un “fournit” le contrat, l’autre le “consomme”). C’est l’opposé d’un réseau plat classique où tout communique par défaut sauf restriction explicite — ACI inverse la logique par sécurité.

Pourquoi le test échoue “après” un déploiement réussi est la leçon la plus importante du TP : un déploiement technique qui réussit (les objets sont bien créés) ne garantit pas que le comportement métier attendu (ici, la communication entre App et DB) fonctionne réellement. C’est pour ça qu’un post-check métier, comme celui écrit ici, est indispensable — le même principe que les post-checks Ansible du TP11, appliqué cette fois à une API cloud-managed.

Validation

- ☐ python scripts/deploy_aci_config.py déploie le tenant complet (VRF, bridge domain, AP, 2 EPG, contrat) en un seul appel
- ☐ La relecture du tenant confirme sa présence
- ☐ La vérification métier confirme qu’EPG_App peut consommer EPG_DB via le contrat
- ☐ Retirer la relation fvRsCons fait échouer la vérification métier avec un message clair, sans casser le déploiement technique lui-même
- ☐ Une pull request <prenom>-<nom>-jour5-tp17 → dev est ouverte